Chapter 3: **Related Work and Ideas**

Much research has been carried out in the fields and areas that are relevant to this project. This chapter outlines literature in the area of cloud computing, and subsequently the motivation for edge computing and MEC. A comparison of the existing ways to simulate a MEC environment is outlined, with reference to past UCD final year projects, followed by a review of different approaches to the scheduling of parallelisable tasks, not only in MEC systems but in general. Finally, research relating to the offloading of tasks due to local constraints is detailed.

3.1 Cloud Computing

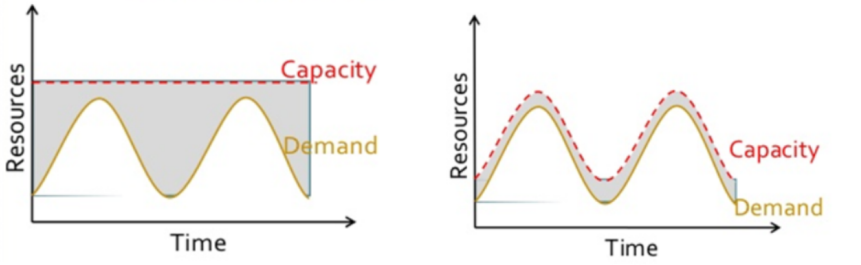
The National Institute of Standards and Technology (NIST) [1] defines cloud computing as follows:

> *Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.*

A major benefit that cloud computing brings to an organisation is the elasticity with which hardware resources can be provisioned and released [1]. An application can harness the powerful computation of large data centre servers without the need to purchase or house it, which is particularly useful when these resources are only needed periodically.

The pay-as-you-go pricing model is attractive in such cases. Despite costing more per-computation, the overall cost is still less than that of the organisation purchasing and running powerful servers themselves, which could remain under-utilised for long periods [2, 3]. Instead, cloud providers run many servers, dynamically allocating computing resources according to the combined demands of multiple users. Figure 3.1 illustrates that this larger volume allows resources to be allocated much more flexibly, and minimise the amount of wasted energy.

In 2014, 17.8% of EU enterprises were purchasing cloud services; eleven years later, in 2025 the figure was 52.7% [4]. However, for time-sensitive tasks, the physical distance between end users and the cloud, as well as network delays, become bottlenecks which edge computing aims to solve.



(a) Traditional Resource Allocation (b) Elastic Resource Allocation

Figure 3.1: Having large amounts of resources available can lead to overprovisioning; cloud computing's elasticity combines multiple services so supply can always stay close to demand [3]

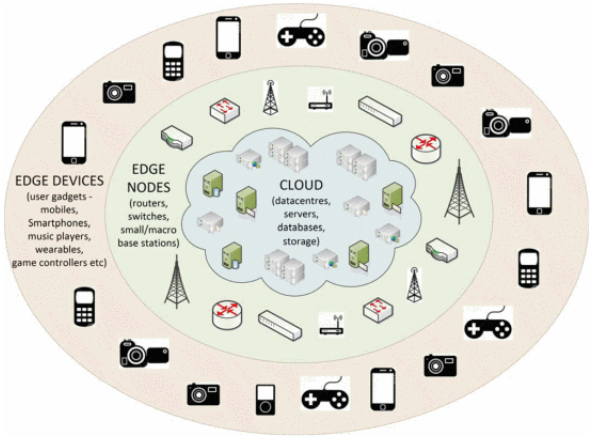


Figure 3.2: Edge devices and edge nodes (or "edge servers") in relation to the cloud [6]

3.2 Edge Computing

Edge computing is a paradigm that aims to solve the aforementioned issues in cloud computing. Sometimes referred to as *fog computing*, the general idea is that substantial computing resources are placed at the "edge" of the Internet, close (in terms of physical distance and/or low latency) to the end users [5], as can be seen in Figure 3.2. An edge server can process both *downstream data* (originating from cloud services) and *upstream data* (originating from end devices).

The decrease in network delay, primarily due to a decrease in physical distance or the number of network hops, is a primary benefit of edge computing. For small tasks, computing them at the edge can be significantly faster (overall) than computing at the cloud. However, unlike traditional cloud computing, a typical edge server cluster is significantly less powerful [7] and may increase task execution time for computationally expensive tasks. Because of this, the overall completion time of processing a large task can still be faster on the cloud, despite its longer round-trip time (RTT). This trade-off, and deciding which tasks get done at the edge, versus which get done on the cloud, is a central research problem in edge computing [8].

3.3 Multi-access Edge Computing (MEC)

Previously known as *Mobile Edge Computing* [9], multi-access edge computing is one specific implementation of the edge computing paradigm. It places edge servers within mobile and wireless access networks. Typically, a MEC system can be conceptually thought of as being made up of three layers: user devices (e.g. mobile phones) at the "edge" of the system, edge servers at a base station or access point, and the central cloud server at the "core" [10].

Although some implementations have the middle layer consisting of multiple sub-layers and *proxy servers* [11, 12], this project adopts a simpler structure as seen in Figure 3.3. This abstraction allows focus to be directed at the scheduling of tasks across the three layers, rather than explicitly modelling low-level networking infrastructure that otherwise can be defined by a simple parameter. The idea that an edge server's RTT will be much lower than the cloud's, combined with the cloud's greater computational power, creates the central scheduling problem that motivates this project.

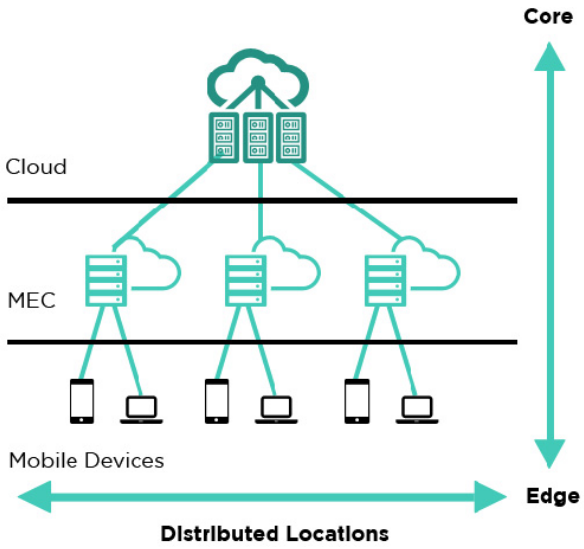


Figure 3.3: A simplified three-layer MEC architecture [10]

3.4 Existing MEC Simulators

MEC presents a wide range of research challenges, including (but certainly not limited to) task scheduling, as will be investigated by this project. To test or verify a solution to a MEC problem, simulation of that problem's processes is a necessity. One useful paper for this section was Kunde et al.'s "Comparison of simulators for fog computing" [13]. There is a collection of simulators and frameworks that are specifically for MEC. In the past, UCD final year projects in this area have tended to use the *iFogSim* simulator, or its successor *iFogSim2*.

3.4.1 iFogSim2

With *CloudSim* and *iFogSim* as its so-called progenitors, *iFogSim2* [14] was built by the same group of researchers in Melbourne that first developed the original *CloudSim*. It describes itself as:

> *A Toolkit for Modeling and Simulation of Resource Management Techniques in Internet of Things, Edge and Fog Computing Environments*

Written in Java, the simulator offers a breadth of pre-defined classes to build a fog computing environment from. Its vast array of potential use-cases comes with the caveat of a steep learning curve, and a tendency to produce errors and bugs. On the former, one would expect an object-oriented edge computing simulator to come pre-packaged with simple classes that intuitively map to the real world entities within a MEC system, for example, classes like `EdgeServer`, `CloudServer`, `Task`, `Scheduler`, etc. that would have constructors requiring very few arguments. Instead, most classes have constructors requiring numerous parameters, including those unrelated to the intended simulation. Some parameters are often themselves objects that need instantiation. For example:

```
FogDevice(String name, FogDeviceCharacteristics characteristics,
    VmAllocationPolicy vmAllocationPolicy, List<Storage> storageList,
    double schedulingInterval, double uplinkBandwidth,
    double downlinkBandwidth, double uplinkLatency, double ratePerMips)

FogDeviceCharacteristics(String architecture, String os, String vmm,
    final Host host, double timeZone, double costPerSec, double costPerMem,
    double costPerStorage, double costPerBw)
```

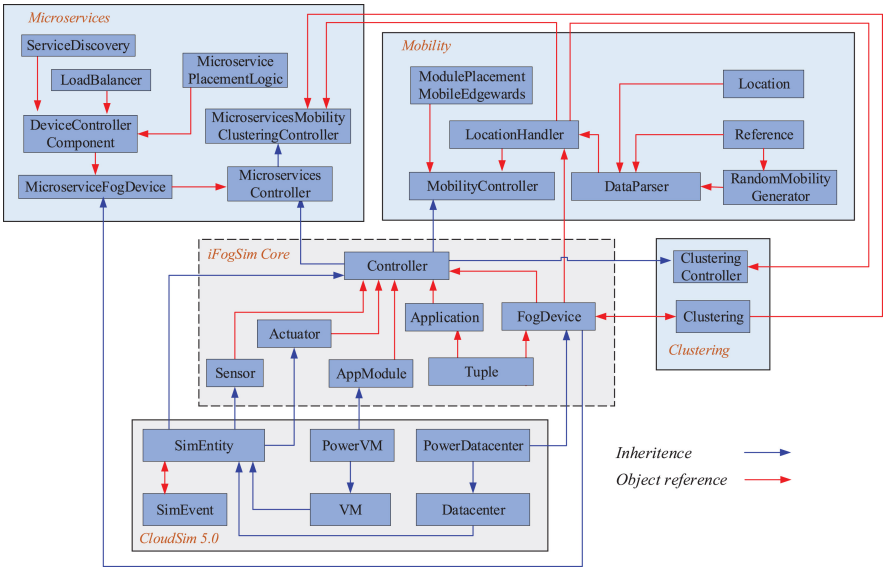


Figure 3.4: Class diagram of iFogSim2, illustrating its architecture [14]

To add to this, the meaning of each class in the system is unintuitive, and the hierarchy of the entire system (a substantial portion of which must be understood to create any simulations) is very complex, as can be seen in Figure 3.4. The simplest example programs supplied on the official GitHub repository span almost 300 lines of code each. This is in part due to the verbose nature of the Java programming language, but is primarily because of the bloated design of *iFogSim2*.

Since its major update and integration with the original *iFogSim* almost two years ago, the GitHub repository containing the source code for the simulator (the primary way to download the software and use it) has not received any updates. Previous students who used *iFogSim2* for their final year projects encountered major bugs that served as large obstacles in their project, diverting development effort away from the novel aspects of their projects. One example is James Finn:

> *The default migration process in iFogSim2 contains several logical limitations in how microservices are handled when users experience mobility events. [...] These issues can result in an incorrect system state where users do not have access to the necessary microservices; therefore, tuples may fail to complete the intended processing loop and prevent the application from executing correctly. [15]*

In addition to this, Cian Boyle found that the simulator had no way to test different fine-grained scheduling algorithms. The existing `FogDevice` class was also not compatible with his goals. Because of this, he had to build an extension to the framework before actually implementing and testing his scheduling algorithm.

> *iFogSim's current implementation includes an inbuilt* `FogDevice` *class which can be used to represent a physical or virtual computing node within a MEC environment. It models complex behaviours relating to microservice placement and mobility which were not necessary for this study and previous implementations have made it difficult to have fine-grained control over task execution. [16]*

In essence, using the *iFogSim2* simulator, at least for problems at the same level of abstraction as this project, will likely result in as much programming as creating a simpler simulator from scratch.

3.4.2 EdgeCloudSim

The *EdgeCloudSim* [17] simulator also builds upon the existing *CloudSim* framework. However, unlike *iFogSim2*, it is not made by the original *CloudSim* developers. It was created specifically to address the concept of *mobility* in MEC, which can be described as:

> *A user's movement may result in disconnection from the old cloudlet [edge server] and reconnection to the new cloudlet, bringing additional task migration overhead. [18]*

The creators of *EdgeCloudSim* discuss in their research paper how *iFogSim* failed to properly model mobility in their simulator, something Finn also observed [15]. It is worth noting that this was the motivation for the creation of *EdgeCloudSim* in 2017, but such functionality does now exist in *iFogSim2* [14].

> *iFogSim allows the definition of the location of devices getting service from the fog servers. However, this information is static and it is not updated by any mobility model. The network model in iFogSim is over simplistic where a fixed delay is assigned to links ignoring the effect of network load on the operation. [19]*

Mobility and network modelling are some of the only areas in which *EdgeCloudSim* can be seen as superior to *iFogSim*, and otherwise they share all the same undesirable qualities (at least for the implementation of this project) as discussed in subsection 3.4.1, such as a steep learning curve, an overly complex class hierarchy, and extremely verbose logging. These issues can be seen in the official tutorials for the simulator. The simulators inherit these problems from the original *CloudSim* simulator, which both of them extend. Therefore, the *EdgeCloudSim* simulator is not well suited to this project since it is not related to the issue of mobility.

3.4.3 EdgeSimPy

Unlike the previous two simulators, which extended *CloudSim* using Java, *EdgeSimPy* [20] instead uses Python. Despite its name, it is not based on the *SimPy* framework (see section 3.5), and the two are completely unrelated. The simulator is built on *Mesa*, which is an agent-based modelling (ABM) framework. Simulated entities are represented as *agents*, which interact and update their states as the simulation progresses [21].

EdgeSimPy simulates all physical entities in a MEC system, such as the servers, the end users themselves, the network infrastructure, etc. This is useful for some scenarios and allows the simulator to be used for a broad set of research problems. It allows programmers to operate at a level of abstraction significantly higher than previously discussed *CloudSim* based simulators, however, a meaningful amount of the network topology must be explicitly defined by the programmer.

Since the simulator is agent-based (as opposed to a DES, see section 3.5), it operates a constantly ticking clock, for example, where one tick corresponds to 1 ms. Then the different parts of the system, such as end users, edge servers, network connections, etc. are all implemented as *agents*. They each complete simulated work, possibly at the same time, for a duration with respect to the global ticking clock.

It provides models of heterogeneous computing infrastructure, network communication, and application services, with support for custom service placement and migration algorithms. This initially makes it a much better candidate than any other simulator. However, the principal focus of this project is the scheduling of parallelisable tasks across local devices, edge servers, and cloud servers. EdgeSimPy can be extended to support such scenarios, but its existing abstractions primarily concern

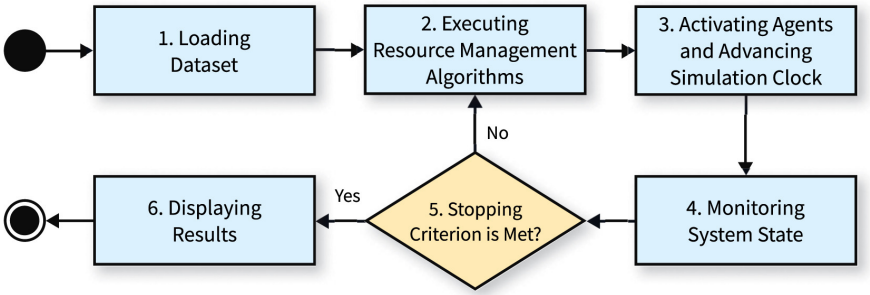


Figure 3.5: *EdgeSimPy*'s simulation workflow [20]

the placement and management of application services rather than the execution and partitioning of individual computational tasks. In particular, its lack of support for partitioning tasks into parallel subtasks mean that substantial additional functionality would be required.

3.5 SimPy

The previous section discussed existing simulators that are purpose-built for MEC simulations. They were all designed to address a broad range of issues in the MEC field. However, this meant that implementing simulations that will be in the narrow scope of this project would require significant changes to the existing software. So, a custom-built simulator is needed for this project.

> *SimPy is a process-based discrete-event simulation framework based on standard Python. [22]*

SimPy is not a MEC simulator. In fact, it is not a simulator at all. It is a Python framework that can be used to design one's own discrete event simulator (DES), rather than starting from scratch. When working with *discrete events*, this means that the system state changes at discrete points in simulated time. For example, if $time = 100$ and the next event is scheduled to occur at $time = 150$, then the engine can advance directly to $time = 150$ rather than simulating all fifty ticks in between unnecessarily. This is a different, but not mutually exclusive, model to the agent-based simulator in subsection 3.4.3.

The *SimPy* framework handles things like the simulation clock and event scheduling, but leaves much of the simulator design up to the programmer. The documentation lists example applications of the framework as simulating a "car wash", "gas station refuelling", a "machine shop", and more.

The reuse of existing software is a well-established practice [23], allowing developers to avoid unnecessarily reimplementing existing functionality. Accordingly, the development of a custom simulator does not necessarily imply implementing every component from scratch. By using SimPy, the discrete-event simulation functionality can be reused while the computational models and scheduling algorithms specific to this project can be implemented separately.

However, a distinction must be made between extending software through its existing interfaces and modifying its underlying implementation. The Open–Closed Principle [23] advocates that software entities should be "open for extension" but "closed for modification". Modifying existing code may introduce unintended interactions with other components that depend on their established behaviour. This is why adapting existing simulators' computational models to support the parallel execution of tasks across local, edge, and cloud resources is an issue. By contrast, using SimPy allows the required functionality to be developed through existing interfaces, without altering the underlying discrete-event simulation engine.

SimPy allows for the representation of multiple activities happening simultaneously. This is particularly useful for the parallel aspect of this project, which will showcase the speedup of executing some of a task's constituent parts simultaneously versus executing the entire task serially.

The capacity of local processors, edge servers, and the cloud server can all be represented by the `Resource` class. This comes with request queues and resource allocation mechanisms built in, and will allow for MIPS-based execution of tasks and subtasks.

Another major benefit of *SimPy* for this project is the way it handles process synchronisation. Supposing a task has been divided into one set to be executed locally and one for the edge, the parent process will only resume once all subtasks have finished. This will be important when discussing *synchronisation constraints*, the idea that subtasks have a collective completion time determined by the slowest one (see section 3.6). Conceptually, code like this can be written:

```
local = env.process(execute_locally(125))  # 125 subtasks to execute locally
edge = env.process(execute_on_edge(25))    # 25 to execute on the edge server
yield env.all_of([local, edge])            # resumes when full task is done
```

3.6 Parallelisable Task Scheduling

The referenced papers in this section (and in Section 3.7) use slightly different language (such as a *job* being made up of *tasks*), so to avoid confusion this discussion will use a consistent terminology of each *task* being made up of smaller *subtasks*, all of which can be (given sufficient hardware resources) executed in parallel, independently of each other.

The scheduling of parallel subtasks is a research problem that is certainly not unique to the area of edge computing. Shafiee and Ghaderi [24] investigated parallel subtask execution internally within one large cluster. An example application they used was *MapReduce*. Each stage (e.g. the *Map* stage) can be thought of as a task, and each worker (e.g. *mapper*) can be thought of as a subtask. The paper designs scheduling algorithms with theoretical guarantees under three main constraints:

Packing Constraint: Each machine can process some number of tasks at the same time. This maps directly to this project, in which there are three "machines": a user device, an edge cluster, and the cloud. Each one consists of *processors* that can execute subtasks in parallel

Synchronisation Constraint: Subtasks that belong to the same task have a "collective completion time", determined by the slowest subtask. Again, this is applicable to this project, where it will be necessary to compare a task's overall completion time when parallelised against its serial execution.

Placement Constraint: A subtask's execution time is machine-dependent. The speed at which machines in their cluster could read/write to disks depended on proximity. With this project, the RTTs and computational power of each "machine" represent this same constraint.

Their algorithms prioritise *time* as the variable to be optimised, which seems appropriate to replicate in this project. They consider memory as their 1-dimensional "resource requirement". Whilst it is important to reduce multi-dimensional problems down to very few dimensions (by fixing all other parameters) [25], it seems like for this project that *computational demand* would be a more suitable and natural "resource requirement".

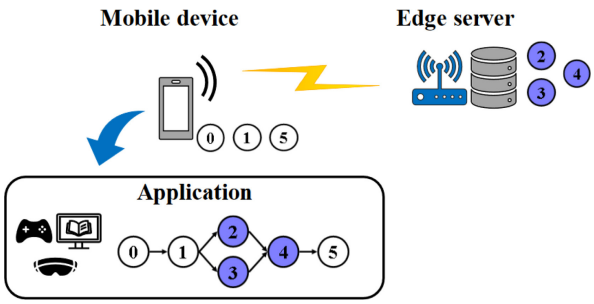


Figure 3.6: DAG representing dependencies between subtasks [27]

Finally, they describe differences between *preemptive* and *non-preemptive* scheduling. Preemptive scheduling means that a subtask can be paused on a processor. This can be to allow a higher-priority subtask to execute on that processor first and/or to migrate a subtask to another processor. In any case preemption is a prerequisite for mid-execution migration. In non-preemptive scheduling, once a subtask has started execution on one processor, it cannot be paused (and hence, definitely cannot be migrated), and must completely finish before anything else can use that processor. Preemption comes with some overhead. This project should investigate both types when proposing algorithms.

3.7 Task Offloading Due To Local Constraints

The most commonly investigated issue in MEC has been task offloading [26]. However, this is usually in the context of an edge server offloading work to the cloud. There is less literature in the area of offloading from a local device to the edge/cloud, and doing so because of local device constraints. In recent times, local devices have become increasingly more powerful. This means that certain subtasks can be executed locally with acceptable delay. This may save on overall latency despite the fact that edge and cloud servers are still generally much more powerful than local devices, as it can avoid the RTT involved in offloading a subtask.

Gabriel et al. [27] created an algorithm for scheduling tasks across the local device and an edge server. They used a directed acyclic graph (DAG) to represent dependencies between subtasks. Subtasks that don't have a dependency between them (e.g. tasks 2 and 3 in Figure 3.6) can be executed in parallel, but in general, tasks must wait until their predecessors are completely finished before they can start. Modelling dependencies between subtasks is one of the exceptional deliverables of this project.

In their model, every task has a subtask with no incoming edges (the *initial task*) and one with no outgoing edges (the *ending task*). Both must take place on the local device. This is a consideration of realistic parallelisable tasks, where results may need to be combined before presenting a result to the user. This consideration will need to be observed when dependencies are explored.

Time-sharing is used, which significantly complicates latency calculations, while also making offloading decisions much harder since processor speed cannot be trusted. One questionable aspect of their scheduling is that as soon as a subtask is "ready to start" (i.e. all its predecessors have finished), it is placed onto a processor immediately, regardless of the system state. Considering that every subtask is assumed to be non-preemptive, immediately starting all subtasks means that migration to faster processors is not possible. This wouldn't be a major issue if time-sharing was not used, because the speed of a given processor would instead be fixed. These features will not be applied in this project.

Bibliography

1. Mell, P. & Grance, T. *The NIST Definition of Cloud Computing* NIST Special Publication 800-145 (National Institute of Standards and Technology, Gaithersburg, MD, 2011).
2. Uslu, A. & Özer, A. H. A Multidimensional Virtual Resource Allocation Framework With Energy-Aware Physical Resource Mapping for Green Cloud Computing. *Concurrency and Computation: Practice and Experience* **37,** e70039 (2025).
3. Fourati, M. H., Marzouk, S. & Jmaiel, M. Cloud Elasticity of Microservices-Based Applications: A Survey. *Concurrency and Computation: Practice and Experience* **37,** e8329 (2025).
4. Eurostat. *53% of EU Enterprises Used Paid Cloud Services in 2025* Feb. 2026. https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20260203-1.
5. Hong, J. *et al. Internet of Things (IoT) Edge Challenges and Functions* RFC 9556. Apr. 2024. https://www.rfc-editor.org/info/rfc9556/.
6. Varghese, B., Wang, N., Barbhuiya, S., Kilpatrick, P. & Nikolopoulos, D. S. Challenges and Opportunities in Edge Computing. In *2016 IEEE International Conference on Smart Cloud (SmartCloud)* 20–26 (2016).
7. Mann, Z. Á. Notions of Architecture in Fog Computing. *Computing* **103,** 51–73 (2021).
8. Avan, A., Azim, A. & Mahmoud, Q. H. A State-of-the-Art Review of Task Scheduling for Edge Computing: A Delay-Sensitive Application Perspective. *Electronics* **12.** ISSN: 2079-9292 (2023).
9. European Telecommunications Standards Institute. *Mobile Edge Computing (MEC); Terminology* ETSI GS MEC 001 V1.1.1 (ETSI, Mar. 2016). https://www.etsi.org/deliver/etsi_gs/MEC/001_099/001/01.01.01_60/gs_mec001v010101p.pdf.
10. Abbas, N., Zhang, Y., Taherkordi, A. & Skeie, T. Mobile Edge Computing: A Survey. *IEEE Internet of Things Journal* **5,** 450–465 (2018).
11. Lee, J. & Lee, J. Hierarchical Mobile Edge Computing Architecture Based on Context Awareness. *Applied Sciences* **8.** ISSN: 2076-3417 (2018).
12. Yu, H., Liu, J., Hu, C. & Zhu, Z. Privacy-Preserving Task Offloading Strategies in MEC. *Sensors* **23.** ISSN: 1424-8220 (2023).
13. Kunde, C. & Mann, Z. Á. Comparison of simulators for fog computing. In *Proceedings of the 35th Annual ACM Symposium on Applied Computing* 1792–1795 (Association for Computing Machinery, Brno, Czech Republic, 2020). ISBN: 9781450368667.
14. Mahmud, R., Pallewatta, S., Goudarzi, M. & Buyya, R. iFogSim2: An extended iFogSim simulator for mobility, clustering, and microservice management in edge and fog computing environments. *Journal of Systems and Software* **190,** 111351. ISSN: 0164-1212 (2022).
15. Finn, J. *HEPA-M: Hierarchical Energy- and Path-Aware Microservice Migration Framework* Bachelor's thesis (University College Dublin, Apr. 2026).
16. Boyle, C. *PATS: A Priority-Aware Task Scheduling Algorithm for Multi-Access Edge Computing* Bachelor's thesis (University College Dublin, Apr. 2026).
17. Sonmez, C., Ozgovde, A. & Ersoy, C. EdgeCloudSim: An environment for performance evaluation of edge computing systems. *Trans. Emerg. Telecommun. Technol.* **29** (Nov. 2018).
18. Zheng, M., Li, J., Hu, Y., Xiao, H. & Hu, Z. Tconns: a novel time-varying context-aware offloading strategy for mobile edge computing. *EURASIP Journal on Wireless Communications and Networking* **2024** (Jan. 2024).

19. Sonmez, C., Ozgovde, A. & Ersoy, C. EdgeCloudSim: An environment for performance evaluation of Edge Computing systems. In *2017 Second International Conference on Fog and Mobile Edge Computing (FMEC)* 39–44 (2017).

20. Souza, P. S., Ferreto, T. & Calheiros, R. N. EdgeSimPy: Python-based modeling and simulation of edge computing resource management policies. *Future Generation Computer Systems* **148,** 446–459. ISSN: 0167-739X (2023).

21. Kazil, J., Masad, D. & Crooks, A. Utilizing Python for Agent-Based Modeling: The Mesa Framework. In *Social, Cultural, and Behavioral Modeling* (eds Thomson, R., Bisgin, H., Dancy, C., Hyder, A. & Hussain, M.) 308–317 (Springer International Publishing, Cham, 2020). ISBN: 978-3-030-61255-9.

22. SimPy Development Team. *SimPy 4.1.2 Documentation* https://simpy.readthedocs.io/en/4.1.2/ (2026).

23. Meyer, B. *Object-Oriented Software Construction* 2nd ed. ISBN: 9780136291558 (Prentice Hall, 1997).

24. Shafiee, M. & Ghaderi, J. Scheduling Parallel-Task Jobs Subject to Packing and Placement Constraints. *CoRR* **abs/2004.00518.** arXiv: 2004.00518 (2020).

25. Chipman, H. & Bingham, D. *Let's practice what we preach: Planning and interpreting simulation studies with design and analysis of experiments* 2021. arXiv: 2111.13737 [stat.ME].

26. Akhlaqi, M. Y. & Mohd Hanapi, Z. B. Task offloading paradigm in mobile edge computing-current issues, adopted approaches, and future directions. *Journal of Network and Computer Applications* **212,** 103568. ISSN: 1084-8045 (2023).

27. de Queiroz, G. F., de Rezende, J. F. & Barbosa, V. C. A flexible algorithm to offload DAG applications for edge computing. *Journal of Network and Computer Applications* **222,** 103791. ISSN: 1084-8045 (2024).